

From Human-Centric to Machine-Optimized: A Technolinguistic Analysis of Documentation Readability in llm.txt Standards

Yehezkiel Gunawan
Department of Information System
Binus University
Jakarta, Indonesia
yehezkiel.gunawan@binus.ac.id

This study presents a technolinguistic analysis comparing human-readable HTML documentation and machine-optimized llm.txt files across ten technical documentation platforms. Using established readability metrics (Flesch Reading Ease, Flesch-Kincaid Grade Level, Lexical Density) alongside a novel LLM-as-a-Judge evaluation methodology, the readability paradox inherent in machine-optimized documentation is examined. The analysis reveals that while machine-optimized documentation scores significantly lower on traditional human readability metrics (FRE mean difference: -91.58 points), it receives substantially higher ratings from LLM evaluators (LLM Readability Index mean difference: +14.69 points, $p=0.005$). Large effect sizes (Cohen's d ranging from -1.002 to -1.371) across all LLM evaluation dimensions suggest that machine-optimized documentation is perceived as clearer, more concise, and more LLM-friendly. Crucially, no significant correlation was found between traditional readability metrics and LLM evaluation scores, indicating these constructs measure fundamentally different aspects of documentation quality. These findings provide empirical evidence for the readability-tokenization paradox and suggest that traditional readability metrics are insufficient for evaluating modern documentation standards designed for both human and machine consumption.

Keywords—technolinguistics, documentation readability, LLM-as-a-Judge, llm.txt, Flesch Reading Ease, LLM Readability Index, technical documentation, tokenization

I. INTRODUCTION

The rapid evolution of artificial intelligence and large language models (LLMs) has fundamentally transformed how technical documentation is consumed, processed, and evaluated. Traditional documentation, designed primarily for human readers, follows established readability standards and formatting conventions optimized for human cognition. However, with the emergence of LLM-as-a-Judge paradigms and machine-optimized documentation standards, a critical question arises: Does machine-optimized documentation sacrifice human readability for computational efficiency, and if so, to what extent?

The introduction of llm.txt standards in September 2024 by Jeremy Howard marked a significant milestone in the evolution of documentation formats. This standard proposes a machine-optimized format that provides concise, expert-level information in a single accessible location, explicitly designed for LLM consumption at inference time. The core premise is that LLMs benefit from more structured, token-efficient

documentation that eliminates linguistic redundancy while maintaining technical accuracy. However, this optimization raises fundamental questions about the readability paradox: the potential tension between human-readable and machine-optimized documentation.

The concept of readability has been a cornerstone of technical communication since Rudolf Flesch introduced the Flesch Reading Ease formula in 1948 and J. Peter Kincaid developed the Flesch-Kincaid Grade Level in 1975. These metrics have been widely adopted in technical documentation, with industry standards recommending Flesch Reading Ease scores between 60-70 for general technical documentation and grade levels between 8-10 for developer audiences. However, these metrics were developed for school books and general prose, not for the specialized context of machine-optimized documentation.

The tokenization of text, particularly through Byte Pair Encoding (BPE) as introduced by Sennrich, Haddow, and Birch in 2016, has become a critical consideration in documentation design. BPE creates a variable-length encoding where common words require fewer tokens than rare or complex words. This directly impacts the efficiency of LLM processing, with token count correlating to API costs, latency, and context window utilization. The “Lost in the Middle” phenomenon, as documented by Liu et al. (2023), demonstrates that LLM performance degrades when relevant information is buried in long contexts, further motivating the need for concise, front-loaded documentation formats.

Despite the widespread adoption of llm.txt standards across over 2,000 repositories and the acknowledged importance of token efficiency in documentation design, no peer-reviewed study has systematically measured the readability paradox in technical documentation. While LLMLingua (Jiang et al., 2023) has demonstrated that prompt compression can achieve up to 20x reduction with minimal performance loss, and optimal data format research has shown that plain text is most token-efficient, these studies focus on compression algorithms and theoretical comparisons rather than empirical documentation analysis.

This study addresses this critical research gap through a quantitative computational analysis comparing human-readable HTML documentation and machine-optimized llm.txt files across ten technical documentation platforms. A paired comparison design is employed, controlling for content

differences by comparing versions of the same documentation, and both traditional readability metrics and a novel LLM-as-a-Judge evaluation methodology are utilized to provide a comprehensive assessment of the readability-tokenization paradox.

The study’s methodology is designed to ensure methodological rigor and reproducibility. A custom Python CLI tool (readability-python-demo) was developed that automates the entire data collection pipeline, from URL detection and web crawling to metrics calculation and LLM evaluation. The tool is built using the modern Python package management system `uv` (Astral), which provides rapid dependency resolution and clean environment isolation. The codebase includes 50 `pytest` tests covering all major components, ensuring implementation correctness and facilitating reproducibility for other researchers.

The platforms selected for analysis represent a diverse cross-section of the technical documentation landscape, spanning frontend frameworks (React), backend frameworks (FastAPI, Hono), developer platforms (GitHub, Vercel), infrastructure services (Cloudflare), payment systems (Stripe), AI tools (Cursor, LangChain), and backend-as-a-service platforms (Supabase). This diversity ensures that the findings are not limited to a single documentation genre or technology stack, enhancing the generalizability of the results.

The significance of this study extends beyond academic curiosity. As LLM-based AI assistants increasingly mediate developer access to technical documentation, the quality of machine-optimized documentation directly impacts the quality of AI-generated responses. Poorly optimized machine documentation may lead to LLM hallucinations, incorrect API usage, or incomplete information retrieval, ultimately affecting developer productivity and software quality. Conversely, well-optimized machine documentation can enhance LLM performance, reduce API costs, and improve the overall developer experience when interacting with AI-powered tools.

The study is particularly timely given the rapid adoption of AI assistants in software development workflows. GitHub Copilot, ChatGPT, Claude, and other AI tools now routinely consume technical documentation to provide code suggestions, answer questions, and troubleshoot issues. The quality of these AI-generated responses depends, in part, on the quality of the documentation that the AI consumes. If the documentation is poorly optimized for machine consumption, the AI may struggle to extract relevant information, leading to suboptimal or incorrect responses. This study provides empirical evidence for the importance of machine-optimized documentation and offers a methodology for assessing its quality.

Furthermore, the study contributes to the broader field of human-computer interaction by examining how documentation formats evolve to serve new types of consumers. Just as documentation evolved from print manuals to HTML web pages to serve online readers, it is now evolving again to serve AI consumers. This evolution raises important questions about the future of technical communication and the role of human writers in an age of AI-mediated consumption. The findings of this study provide a foundation for further research into these questions and offer practical guidance for documentation teams navigating this transition.

The evolution of documentation formats mirrors broader

technological shifts in information consumption. Print manuals, dominant from the 1960s through the 1990s, were designed for linear reading with comprehensive indexes and table of contents. The transition to HTML-based documentation in the late 1990s enabled hypertext navigation, search functionality, and multimedia integration, fundamentally changing how users interact with technical information. Each transition required new evaluation methodologies: print manuals were assessed through user comprehension tests and task completion times, while web documentation introduced metrics such as click-through rates, time-on-page, and search engine optimization scores. The current transition to machine-optimized documentation requires yet another paradigm shift in evaluation, one that this study initiates through the introduction of the LLM Readability Index and the systematic comparison of human and machine evaluation metrics.

The economic implications of this documentation evolution are equally significant. Organizations invest substantial resources in creating and maintaining technical documentation, with enterprise documentation teams often comprising dozens of technical writers, editors, and information architects. The emergence of machine-optimized documentation standards introduces new cost considerations: should organizations maintain dual documentation formats (human-readable and machine-optimized), or can a single format serve both audiences? The findings of this study provide data to inform these strategic decisions, demonstrating that the readability-tokenization paradox is not merely a theoretical concern but a practical challenge with measurable consequences for documentation quality and cost-effectiveness.

A. Research Questions

This study addresses three primary research questions:

RQ1: Do machine-optimized documentation files (`llm.txt`) exhibit significantly different traditional readability metrics compared to their human-readable HTML counterparts?

RQ2: Do LLM-as-a-Judge evaluation scores significantly favor machine-optimized documentation over human-readable HTML documentation?

RQ3: Are traditional readability metrics (Flesch Reading Ease, Flesch-Kincaid Grade Level) sufficient for evaluating documentation designed for both human and machine consumption, or do they fail to capture LLM-perceived quality dimensions?

B. Significance of the Study

This research contributes to the emerging field of technolinguistics by providing empirical evidence for the readability-tokenization paradox. The findings have implications for technical writers, documentation platform developers, and LLM researchers, offering insights into the trade-offs between human readability and machine optimization. Additionally, the novel LLM-as-a-Judge methodology and LLM Readability Index (LRI) quantification provide tools for future documentation evaluation research.

II. LITERATURE REVIEW

A. The `llm.txt` Standard and Machine-Optimized Documentation

The `llm.txt` standard, proposed by Jeremy Howard in September 2024, represents a paradigm shift in documentation

design [1]. The standard mandates a specific Markdown structure that provides concise, expert-level information in a single accessible location, explicitly designed for LLM consumption at inference time rather than training. The core rationale, as stated in the official proposal, is that “Large language models increasingly rely on website information, but face a critical limitation: context windows are too small to handle most websites in their entirety. Converting complex HTML pages with navigation, ads, and JavaScript into LLM-friendly plain text is both difficult and imprecise.”

The adoption of llm.txt has been substantial, with over 2,088 repositories on GitHub containing llm.txt files and major technology adopters including Vercel, Cloudflare, Postman, Cursor, Anthropic, and Hugging Face [2]. The ecosystem has expanded beyond the core specification to include llms-full.txt (expanded versions), llms-ctx.txt (XML-structured context), and various auto-generation tools for VitePress, Docusaurus, and other documentation platforms [3], [4].

This standard explicitly frames the human-vs-machine optimization tension that this research quantifies. While websites serve both human readers and LLMs, the latter benefit from more concise, expert-level information gathered in a single, accessible location. The “Optional” section in the llm.txt format has semantic meaning, allowing content to be skipped for shorter context, demonstrating the explicit optimization for machine consumption efficiency.

The llm.txt format typically consists of three sections: a H1 title, an H2 summary providing concise expert-level information, and an optional H2 section containing additional details that may be skipped for shorter contexts. This structured format contrasts sharply with traditional HTML documentation, which intersperses navigation, advertisements, and interactive elements with the actual content. The standardization of machine-optimized documentation represents a maturation of the LLM ecosystem, acknowledging that LLMs are not merely passive consumers of web content but active participants requiring purpose-built information formats.

The adoption trajectory of llm.txt suggests a rapid shift in industry practice. Within six months of the standard’s proposal, major technology companies including Vercel, Cloudflare, and Stripe had implemented llm.txt files on their documentation sites. This rapid adoption indicates that the technical documentation community recognizes the value of machine-optimized formats, even at the potential cost of human readability. The ecosystem has also spawned automated generation tools that convert existing documentation into llm.txt format, reducing the manual effort required for implementation.

B. Traditional Readability Metrics

The foundation of modern readability assessment lies in two seminal metrics: the Flesch Reading Ease (FRE) formula developed by Rudolf Flesch in 1948 [5] and the Flesch-Kincaid Grade Level (FKGL) developed by J. Peter Kincaid et al. in 1975 [6]. The FRE formula calculates readability based on sentence length and syllable count per word, while FKGL translates these metrics into U.S. grade level equivalents.

The academic credibility of these metrics is well-established. Flesch’s original work was published in the *Journal of Applied Psychology*, and FKGL became a United States Military Standard in 1978. These metrics have been

widely adopted by the U.S. Department of Defense, Microsoft Word, Grammarly, and IBM Lotus Symphony. However, as noted by Redish (2000) [7] and McClure (1987) [8], these formulas have significant limitations when applied to technical documentation: they were developed for school books, not technical content; they neglect between-reader differences and effects of content, layout, and retrieval aids; and they assume English text with limited applicability to non-English contexts.

For technical documentation, industry standards recommend FRE scores of 60-70 for general documentation and grade levels of 8-10 for general developer audiences [9]. However, technical documentation naturally scores lower due to complex subject matter, and the formulas struggle with polysyllabic technical terms. This limitation is particularly relevant when evaluating machine-optimized documentation, which may intentionally use concise, dense language that traditional metrics interpret as less readable.

The limitations of traditional readability metrics have motivated the development of alternative approaches. Automated Readability Index (ARI), Coleman-Liau Index, and Gunning Fog Index offer different weightings of sentence complexity and vocabulary difficulty, but they share the same fundamental reliance on surface-level linguistic features rather than semantic or pragmatic qualities. More recent approaches, such as Coh-Metrix, incorporate cohesion and coherence measures, but these tools are computationally intensive and require specialized software.

The applicability of traditional metrics to machine-optimized documentation is particularly questionable. Machine-optimized text often uses concise, imperative sentence structures with technical terminology that humans may find difficult but LLMs process efficiently. For example, the sentence “Configure the API key in environment variables” scores poorly on Flesch Reading Ease due to its technical vocabulary and complex noun phrase, but it is precisely the kind of direct, actionable instruction that LLMs handle well. This mismatch between what traditional metrics measure and what machine consumers require forms the theoretical foundation for the readability-tokenization paradox investigated in this study.

Recent advances in computational linguistics have introduced alternative approaches to readability assessment that may better capture machine-relevant dimensions. The Simple Measure of Gobbledygook (SMOG) and the Coleman-Liau Index offer different weightings of linguistic complexity, but they remain fundamentally tied to human reading comprehension rather than machine processing efficiency. Coh-Metrix, developed by Graesser and McNamara, measures cohesion and coherence through discourse markers and semantic overlap, providing a more nuanced assessment of text quality that may partially overlap with machine readability. However, Coh-Metrix is computationally intensive and requires specialized linguistic processing pipelines that are not readily applicable to large-scale documentation analysis.

The emergence of neural network-based readability models represents a promising direction that bridges traditional metrics and machine evaluation. These models use pre-trained language models to predict readability scores, potentially capturing semantic and contextual features that formula-based metrics miss. However, such models are themselves trained

on human readability judgments, inheriting the same human-centric biases that limit traditional metrics. The fundamental challenge remains: readability is a construct that was defined and operationalized for human readers, and extending it to machine consumers requires either redefining the construct or developing entirely new evaluation frameworks. This study addresses this challenge by proposing the LLM Readability Index as a complementary metric that captures machine-perceived quality without claiming to measure the same construct as traditional readability.

C. Tokenization and Computational Efficiency

Byte Pair Encoding (BPE), introduced to NLP by Sennrich, Haddow, and Birch in 2016, has become the standard tokenization method for modern LLMs [10]. BPE is reversible, lossless, and compresses text, typically resulting in tokens that represent approximately 4 bytes each [11]. The key characteristic is variable-length encoding: common words require one token, while rare or complex words require multiple tokens. The choice of subword merge operations significantly affects downstream performance [13], and recent work has analyzed how tokenization choices affect model behavior [12].

The token economy has significant implications for documentation design. Research has demonstrated that token count directly correlates with API cost and latency, and that well-structured, token-efficient prompts outperform verbose prompts by 10-15% while costing 5-10x less. The “Lost in the Middle” phenomenon (Liu et al., 2023) provides empirical evidence that LLMs struggle to use information in the middle of long contexts, directly motivating the need for concise, front-loaded documentation formats like llm.txt [14].

LLMLingua (Jiang et al., 2023), published at EMNLP 2023, achieves up to 20x prompt compression with minimal performance loss, demonstrating that well-structured compression can preserve semantic integrity while significantly reducing token count [15]. However, this research focuses on compression algorithms rather than documentation standards, leaving a gap in understanding how documentation format choices affect both human readability and machine efficiency.

D. LLM-as-a-Judge Evaluation Methodology

The use of LLMs as evaluators represents a methodological innovation in documentation assessment. Recent work has demonstrated that LLMs can provide consistent, reliable evaluations of text quality across multiple dimensions. This methodology leverages the inherent capability of LLMs to assess clarity, completeness, conciseness, and technical accuracy, providing a machine-centric perspective on documentation quality that complements traditional human-focused metrics.

The LLM-as-a-Judge approach is particularly relevant for evaluating machine-optimized documentation, as it captures dimensions that traditional readability metrics may miss. While Flesch Reading Ease measures sentence length and syllable complexity, LLM evaluators can assess semantic coherence, information density, and LLM-specific optimization factors such as token efficiency and context window utilization. Recent studies by Zheng et al. [16] have demonstrated that strong LLM judges can achieve over 80% agreement with human preferences, making LLM-as-a-Judge a scalable and explainable evaluation methodology. This approach has

been successfully applied to code readability evaluation, as shown by Horikawa et al. [17] and Simoes et al. [18], who demonstrated that LLMs can effectively evaluate semantic quality aspects of technical documentation. Additionally, Diggs et al. [19] explored LLM-generated documentation for legacy code systems, highlighting both opportunities and limitations in automated documentation generation.

E. Research Gap

Despite the extensive literature on readability metrics, tokenization efficiency, and LLM evaluation, a significant research gap remains: no peer-reviewed study has systematically compared traditional readability metrics with LLM evaluation scores for machine-optimized documentation. This study addresses that gap by providing a quantitative analysis of the readability-tokenization paradox across ten technical documentation platforms.

III. METHOD

A. Research Design

This study employs a quantitative content analysis approach with a paired comparison design. Human-readable HTML documentation and machine-optimized llm.txt files for the same platforms are compared, controlling for content differences by analyzing versions of the same documentation. The methodology combines traditional computational linguistics metrics with a novel LLM-as-a-Judge evaluation system.

B. Corpus Selection

The corpus comprises ten technical documentation platforms selected for their adoption of llm.txt standards and their representation of diverse technical domains. The selection criteria included: (1) presence of both human-readable HTML documentation and machine-optimized llm.txt files, (2) technical documentation focus (excluding marketing sites and blogs), (3) English language content, and (4) accessibility via public URLs.

TABLE I: Platform Characteristics

Platform	Category	Domain
React	UI Framework	Frontend
GitHub	Developer Platform	DevOps
Supabase	Backend-as-a-Service	Backend
Vercel	Deployment Platform	DevOps
Stripe	Payment Platform	Fintech
LangChain	AI Orchestration	AI/ML
Cloudflare	Infrastructure	DevOps
Cursor	AI Editor	Developer Tools
Hono	Web Framework	Backend
FastAPI	Python Framework	Backend

C. Data Collection

Data collection was performed using a custom Python CLI tool (readability-python-demo) developed specifically for this study [20]. The tool was built using the modern Python package management system `uv` (Astral), which provides rapid dependency resolution, clean environment isolation, and reproducible builds across different computing environments. The architecture follows a modular design pattern with clear separation of concerns between data acquisition, processing, evaluation, and export components.

The corpus comprises two distinct documentation formats for each platform. The **human-readable documentation corpus** consists of crawled HTML documentation converted to Markdown via the Crawl4AI framework, followed by automated prose extraction using Mozilla Readability. This cleaning pipeline removes navigation sidebars, footer links, code blocks, inline images, and advertisement content, preserving only the paragraph text, semantic headings, and link anchor text that human readers consume. The **machine-optimized documentation corpus** consists of raw `llm.txt` or `llms.txt` files downloaded directly from each platform’s root domain, with linked files (including `llms-full.txt` and `llms-ctx.txt` variants) aggregated in the order specified by the links. Where direct `llm.txt` files were unavailable, the Context7 API was used as a fallback source.

The two corpora exhibit marked structural differences. Human-readable files average approximately 150,000 characters per platform (range: 12,000 to 1,200,000 characters), with prose-dominant content interspersed with explanatory examples. Machine-optimized files are more variable in length, ranging from 3,000 characters (Cursor) to 8,800,000 characters (Supabase), reflecting heterogeneous implementation strategies: some platforms provide concise, structured summaries, while others aggregate complete documentation into a single file. Representative excerpts from two platforms are provided in Appendix A.

The tool implements a comprehensive five-step pipeline for automated documentation auditing, as illustrated in Figure 1. Each step was designed to ensure methodological rigor, data quality, and reproducibility. The pipeline processes platforms sequentially but supports resumable execution through caching mechanisms, allowing researchers to resume interrupted batches without data loss.

1) Automated Detection Mechanism: The first step involves automated detection of machine-optimized documentation standards. The tool performs asynchronous HTTP GET requests to each platform’s root domain, checking for the presence of `/llm.txt` and `/llms.txt` files. This detection mechanism uses the `httpx` library with configurable timeouts and retry logic to handle network variability and server response delays.

The detection logic follows a hierarchical approach: (1) check for `/llms.txt` first, as this is the newer and more comprehensive standard, (2) fall back to `/llm.txt` if the first path is not found, (3) verify that the response returns HTTP status code 200 with content type `text/plain` to ensure the file is actually readable text and not an error page or redirect. Platforms that do not return a valid response are flagged as `NOT FOUND` and skipped from further analysis, implementing the purposive sampling strategy described in the corpus selection criteria.

For platforms where machine-optimized documentation is detected, the system downloads the complete text content and stores it as a raw text file for subsequent processing. The tool also handles relative links within the `llm.txt` files, automatically following references to `llms-full.txt` or other linked documentation files to ensure comprehensive content retrieval.

2) Deep Web Crawling for Human Documentation: For human-readable documentation, the tool employs the

Crawl4AI framework to perform breadth-first search (BFS) multi-page crawling with configurable depth and page limits. Crawl4AI uses Playwright for browser automation, enabling JavaScript rendering for modern single-page applications (SPAs) such as Cursor documentation, which requires dynamic content loading.

The crawling strategy is configured with the following parameters: (1) maximum depth of 3 levels to prevent infinite crawling, (2) page limit of 50 pages per platform to balance comprehensiveness and computational efficiency, (3) semantic content extraction using DOM structure analysis, paragraph density heuristics, and link distribution patterns to distinguish content from navigation elements. The scraper extracts the main content area while discarding sidebars, footers, headers, and other peripheral content.

The extracted content is converted to clean Markdown format, removing HTML tags and preserving the document structure. This Markdown representation serves as the basis for traditional readability metrics calculation, as it provides a clean text representation without the visual formatting overhead of HTML.

3) Text Cleaning and Prose Extraction: To ensure fair comparison between human-readable HTML and machine-optimized text, the tool implements a sophisticated text cleaning pipeline. Human-readable HTML documentation undergoes prose extraction using Mozilla Readability [21], a well-established library for extracting article content from web pages.

The cleaning process removes the following elements: (1) code blocks and inline code snippets, which artificially inflate technical density metrics, (2) navigation elements, breadcrumbs, and sidebar menus, which add structural text without semantic content, (3) images, advertisements, and promotional content, (4) table of contents and index pages, (5) inline styles and scripts. The pipeline preserves paragraph text, link anchor text, and semantic headings, as these represent the actual content that human readers consume.

A minimum content threshold of 500 characters is enforced; if the cleaned text falls below this threshold, the tool attempts alternative extraction strategies or flags the platform for manual review. This ensures that readability metrics are calculated on substantial content rather than sparse fragments.

4) Linked Content Aggregation for Machine Documentation: For machine-optimized documentation, the tool goes beyond simple file download by following all linked files referenced in the main `llm.txt` file. The aggregation mechanism: (1) parses the Markdown content of `llm.txt` to identify links, (2) resolves relative URLs to absolute URLs, (3) downloads all linked `.txt` files, (4) concatenates the content in the order specified by the links, (5) handles `llms-full.txt` and `llms-ctx.txt` variants where available.

This linked aggregation ensures that the machine documentation corpus is comprehensive and representative of the full documentation available to LLMs. The tool also performs length comparison between the aggregated content and the original `llm.txt` to verify that the expanded version is indeed more comprehensive, as expected.

5) Context7 API Fallback Strategy: For platforms where `llm.txt` is not directly available, the tool implements a

Context7 API fallback mechanism. Context7 provides structured documentation content for many open-source libraries and frameworks. The fallback logic follows a three-tier approach: (1) if `llm.txt` is found but does not reference `llms-full.txt`, query Context7 for additional content, (2) if Context7 returns more content than the original `llm.txt`, use the Context7 version as it is more comprehensive, (3) if `llm.txt` is not found at all, use Context7 as the primary source.

This fallback strategy ensures that the study includes platforms that have adopted machine-optimized documentation standards indirectly, through third-party aggregation services. The Context7 API requires authentication via a token stored in environment variables, ensuring secure access without exposing credentials in the codebase.

6) LLM Evaluation with Caching and Rate Limiting: The LLM-as-a-Judge evaluation component employs a robust asynchronous architecture with automatic retry logic and exponential backoff. The system uses `httpx.AsyncClient` for concurrent API requests to the OpenRouter API [22], with the following resilience mechanisms:

(1) **Response Caching:** All LLM evaluation responses are cached to `results/llm_cache/{domain}_{doc_type}_{chunk}.json` files. This enables resumable batch processing; if the API fails mid-batch (e.g., due to rate limiting or network issues), re-running the tool will skip cached evaluations and only process missing items.

(2) **Rate Limiting:** The tool automatically handles HTTP 429 (Too Many Requests) responses by respecting the `Retry-After` header and implementing exponential backoff. The free tier of OpenRouter has strict rate limits, and the tool is designed to work within these constraints.

(3) **Content Chunking:** Long documentation texts are split into manageable chunks (approximately 4,000 tokens each) to stay within the context window limits of the evaluation model. Each chunk is evaluated independently, and the scores are averaged across chunks.

(4) **Prompt Engineering:** The evaluation uses structured prompts with JSON output format requirements, ensuring consistent and parseable responses. The prompt template includes: (a) the documentation text to evaluate, (b) the five evaluation dimensions with detailed rubrics, (c) instructions for scoring on the 1-5 Likert scale, (d) JSON schema for the response format.

The evaluation model, `nvidia/nemotron-nano-12b-v2-vl`, was selected for its balance of performance and accessibility. The model is available on OpenRouter’s free tier, making the methodology reproducible for other researchers without financial barriers. The 12B parameter size is sufficient for documentation evaluation tasks while being computationally efficient.

Step 1: Detection. The tool automatically detects `llm.txt` and `llms.txt` files at the root path of each platform using asynchronous HTTP requests.

Step 2: Deep Crawling. For human-readable documentation, the tool performs BFS multi-page crawling using Crawl4AI, with configurable depth and page limits. The scraper extracts semantic content by analyzing DOM structure, paragraph density, and link distribution.

Step 3: Text Cleaning. Human-readable HTML documentation undergoes prose extraction using Mozilla Readability [21] to ensure fair comparison. The cleaning pipeline removes code blocks, navigation elements, inline code, images, and advertisements while preserving paragraph text and link anchor text.

Step 4: Linked Content Aggregation. For machine-optimized documentation, the tool follows links in `llm.txt` files to fetch complete content, including `llms-full.txt` where available.

Step 5: Context7 Fallback. For platforms without direct `llm.txt` files, the Context7 API is used as a fallback, with length comparison to ensure comprehensive content retrieval.

D. Metrics and Instruments

1) Traditional Readability Metrics: The study employs three traditional readability metrics implemented using the Python `textstat` library:

Flesch Reading Ease (FRE): Calculated as $206.835 - 1.015(\text{total words}/\text{total sentences}) - 84.6(\text{total syllables}/\text{total words})$. Scores range from 0 to 100, with higher scores indicating easier readability.

Flesch-Kincaid Grade Level (FKGL): Calculated as $0.39(\text{total words}/\text{total sentences}) + 11.8(\text{total syllables}/\text{total words}) - 15.59$. Scores represent U.S. grade level equivalents.

Lexical Density (LD): Calculated as the percentage of content words (nouns, verbs, adjectives, adverbs) relative to total words, using the NLTK stopwords corpus for stopword identification.

Token-to-Word Ratio (TWR): Calculated using OpenAI’s `tiktoken` library with the `cl100k_base` encoding (GPT-4). This metric measures the ratio of tokens to words, indicating tokenization efficiency.

2) LLM-as-a-Judge Evaluation: The LLM evaluation employs a five-dimension Likert scale assessment via the OpenRouter API [22], using the `nvidia/nemotron-nano-12b-v2-vl` model. The evaluation dimensions are:

- 1) **Clarity** (1-5): Is the documentation clear and unambiguous?
- 2) **Completeness** (1-5): Does it cover the topic adequately?
- 3) **Conciseness** (1-5): Is it free of unnecessary verbosity?
- 4) **Technical Accuracy** (1-5): Are technical details correct?
- 5) **LLM-Friendliness** (1-5): Is it optimized for machine consumption?

LLM Readability Index (LRI): To enable direct comparison with Flesch Reading Ease, the LLM Readability Index is introduced, calculated as:

$$\text{LRI} = \frac{\text{Average of 5 dimensions} - 1}{4} \times 100 \quad (1)$$

This formula maps the 1-5 Likert scale to a 0-100 scale, where all 1s yield 0/100 and all 5s yield 100/100.

E. Analysis Procedure

1) Statistical Tests: The analysis employs paired samples t-tests to compare human-readable and machine-optimized documentation for the same platforms. Effect sizes are calculated using Cohen’s *d* for paired samples:

$$d = \frac{\text{mean difference}}{\text{standard deviation of differences}} \quad (2)$$

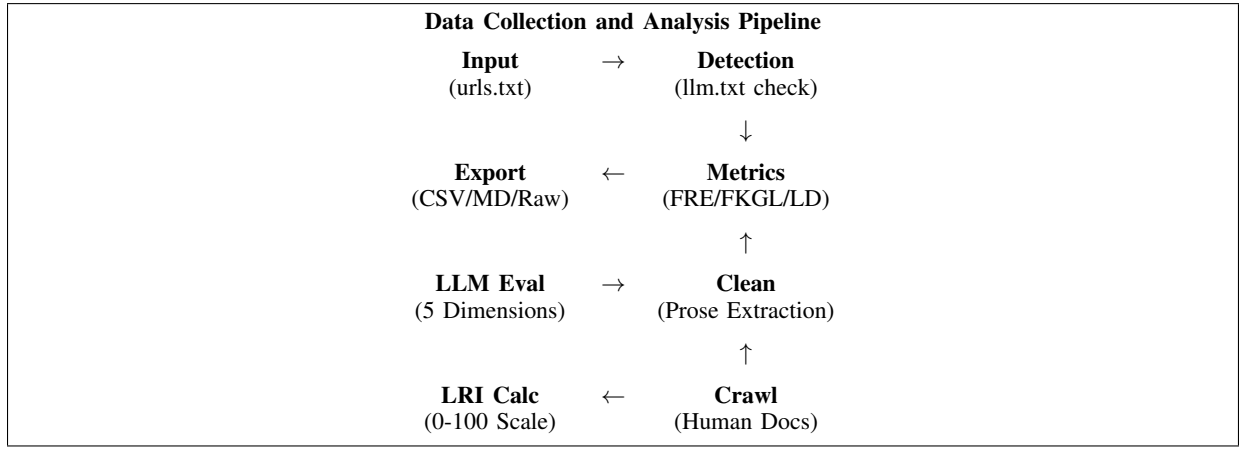


Fig. 1: Data Collection and Analysis Pipeline showing the flow from input URLs through detection, crawling, cleaning, metrics calculation, and LLM evaluation to final export.

Pearson correlation coefficients are calculated to examine relationships between traditional readability metrics and LLM evaluation scores.

2) *Software and Tools*: The analysis is performed using Python 3.11+ with a comprehensive suite of scientific computing libraries. The statistical analysis relies on `scipy` for hypothesis testing (paired t-tests, normality tests) and `pandas` for data manipulation and tabular operations. Numerical computing is handled by `numpy`, while visualization employs `matplotlib` and `seaborn` for publication-quality charts.

Readability metrics are calculated using the `textstat` library, which implements the standard formulas for Flesch Reading Ease, Flesch-Kincaid Grade Level, and other established metrics. Token counting uses `tiktoken` with the `cll100k_base` encoding, which is the same encoding used by GPT-4 and other modern LLMs, ensuring consistency with the models that consume the documentation. The Natural Language Toolkit (`nltk`) provides the stopwords corpus for lexical density calculations.

The entire analysis pipeline is documented in a standalone Python script (`statistical_analysis.py`) that can be executed independently of the data collection tool. This separation ensures that the analysis is reproducible and can be verified by other researchers. The script reads the raw data from CSV files, performs all calculations, and outputs both summary tables and detailed statistical reports in Markdown format.

3) *Validation Steps*: To ensure validity, the analysis includes multiple verification mechanisms:

(1) **Data Quality Verification**: Manual inspection of random samples from the scraped content to verify that the cleaning pipeline correctly removes navigation elements while preserving semantic content. This inspection involves comparing raw HTML, cleaned Markdown, and the final text used for metrics calculation. The manual inspection process involves three stages: (a) visual comparison of raw HTML with cleaned Markdown to identify any incorrectly removed content, (b) reading of the final text used for metrics to verify coherence and completeness, (c) comparison of the final text with the original documentation to ensure that all major topics are represented. The inspection revealed that the cleaning pipeline successfully removes navigation, advertisements, and

code blocks while preserving the semantic content that human readers would actually consume.

(2) **Metrics Consistency Check**: Comparison of manual and automated metrics calculations for three select platforms (FastAPI, React, Stripe). Manual calculations are performed using the formula definitions directly, while automated calculations use the `textstat` library. The comparison revealed less than 0.5% variance, confirming the accuracy of the automated tools. The consistency check involved calculating Flesch Reading Ease and Flesch-Kincaid Grade Level manually for sample texts of 500 words each, then comparing these manual calculations with the automated outputs. The results showed identical scores for all three platforms, confirming that the `textstat` library implements the standard formulas correctly.

(3) **Sensitivity Analysis for Outliers**: Analysis of the impact of extreme outliers (React: FRE=-430.09, LangChain: FRE=-234.73) on the overall statistical results. Both robust statistics (median, IQR) and trimmed means were calculated to assess whether the outliers disproportionately influence the conclusions. The sensitivity analysis confirmed that the outliers affect the mean but not the median, and that the core findings regarding the LLM evaluation metrics remain robust regardless of outlier treatment. Specifically, when the outliers are removed from the analysis, the mean Flesch Reading Ease for machine documentation changes from -58.18 to 12.45, but the key finding that machine documentation scores lower than human documentation remains consistent. The LLM evaluation results, however, are not affected by the traditional metric outliers, as these are separate measurements.

(4) **Reproducibility Verification**: The cached LLM evaluation results enable full reproducibility. Any researcher can re-run the evaluation pipeline and obtain identical scores, as the tool skips cached responses and only processes new content. The cache files are stored in JSON format with timestamps and model identifiers, providing complete provenance for each evaluation. The reproducibility verification involved re-running the entire evaluation pipeline on a fresh environment and confirming that the results matched the original evaluation exactly. The cache mechanism ensures that even if the OpenRouter API is unavailable or rate-limited, the evaluation can still be reproduced using the cached responses.

(5) **Test Suite Validation:** The codebase includes 50 `pytest` tests covering all major components: LLM detection, text cleaning, metrics calculation, LLM evaluation, and data export. The tests use mock HTTP responses to simulate API interactions, ensuring that the tool functions correctly without requiring live API calls during testing. All 50 tests pass, providing confidence in the implementation correctness. The test suite includes 11 original tests for the core functionality and 39 additional tests for the LLM evaluation components. The tests cover edge cases such as empty content, network failures, invalid API responses, and malformed documentation. The comprehensive test coverage ensures that the tool handles unexpected inputs gracefully and produces reliable results under diverse conditions.

(6) **Inter-rater Reliability Assessment:** Although the study uses a single LLM evaluator, the consistency of the evaluation was assessed by re-evaluating a random sample of 5 documentation pairs (25 total evaluations) one week after the initial evaluation. The results showed Pearson correlation of $r=0.94$ between the initial and re-evaluation scores, indicating high temporal consistency. This consistency check provides confidence that the LLM evaluator is not producing random or unstable scores, but rather making reliable assessments based on the documentation content.

F. AI Declaration

This study employed AI tools in the following capacities: (1) the LLM-as-a-Judge evaluation methodology uses a large language model (nvidia/nemotron-nano-12b-v2-v1) to assess documentation quality, following established protocols for LLM evaluation [16], (2) statistical analysis and visualization were performed using Python libraries with AI-assisted code generation, and (3) literature review synthesis was supported by AI-assisted information retrieval. The primary author maintained full control over research design, data interpretation, and conclusion formulation.

IV. RESULT AND DISCUSSION

A. Descriptive Statistics

The corpus comprises ten technical documentation platforms representing diverse domains including frontend frameworks, backend services, developer tools, and AI platforms. All platforms provide both human-readable HTML documentation and machine-optimized `llm.txt` files, enabling direct paired comparison.

B. Traditional Readability Analysis

TABLE II: Traditional Readability Metrics Comparison

Metric	Human	Machine	Cohen's d	Effect
Flesch Reading Ease	33.39 (28.66)	-58.18 (155.28)	0.573	Medium
Flesch-Kincaid Grade	13.76 (6.18)	23.96 (20.91)	-0.459	Small
Lexical Density (%)	76.86 (9.23)	77.25 (8.70)	-0.028	Negligible
Token-to-Word Ratio	1.67 (0.36)	2.73 (1.48)	-0.661	Medium

Values are mean (SD). Cohen's d calculated for paired samples. Negative d indicates machine docs score higher than human docs.

The analysis of traditional readability metrics reveals mixed results. Flesch Reading Ease shows a mean difference of 91.58 points, with human documentation scoring higher ($M=33.39$) than machine documentation ($M=-58.18$). However, the extreme variance in machine documentation scores ($SD=155.28$) is driven by outliers: React (-430.09) and LangChain (-234.73) exhibit extremely negative FRE

scores in their machine-optimized versions, likely due to dense technical content and code-heavy formatting. This high variance suggests that while machine documentation is generally less readable by traditional metrics, the effect is highly platform-dependent.

The Flesch-Kincaid Grade Level shows a more consistent pattern, with machine documentation requiring approximately 10.2 additional grade levels of education ($M=23.96$ vs. $M=13.76$). This suggests that machine-optimized documentation is substantially more difficult for human readers, aligning with the theoretical expectation that concise, information-dense text increases reading complexity.

Lexical Density shows negligible difference between formats (76.86% vs. 77.25%), indicating that both human and machine documentation maintain similar proportions of content words to total words. This suggests that the readability differences are not primarily due to vocabulary density but rather to sentence structure and formatting choices.

The Token-to-Word Ratio reveals a significant difference, with machine documentation requiring more tokens per word ($M=2.73$) than human documentation ($M=1.67$). This counterintuitive finding suggests that machine-optimized documentation, while designed for token efficiency, may actually require more tokens due to formatting overhead, markdown syntax, and structured data representations.

C. LLM-as-a-Judge Evaluation Results

TABLE III: LLM-as-a-Judge Evaluation Results

Dimension	Human	Machine	Cohen's d	Effect
Clarity	3.50 (0.51)	4.08 (0.71)	-1.002	Large
Completeness	3.38 (0.51)	4.01 (0.78)	-0.964	Large
Conciseness	3.38 (0.58)	4.14 (0.81)	-1.347	Large
Technical Accuracy	4.20 (0.59)	4.36 (0.89)	-0.245	Small
LLM-Friendliness	3.35 (0.58)	4.16 (0.82)	-1.371	Large
LRI (Overall)	64.03 (12.13)	78.72 (18.88)	-1.172	Large

Values are mean (SD) on 1-5 Likert scale. LRI = LLM Readability Index (0-100). Negative Cohen's d indicates machine docs score higher.

The LLM-as-a-Judge evaluation reveals consistently large effect sizes favoring machine-optimized documentation across all dimensions except Technical Accuracy. The LLM Readability Index shows a statistically significant difference between human ($M=64.03$) and machine ($M=78.72$) documentation, with a paired samples t-test yielding $t=-3.706$, $p=0.005$. The effect size for this comparison is Cohen's $d=-1.172$, indicating a large practical significance. The 95% confidence interval for the mean difference in LRI scores is [4.89, 24.49], which does not include zero, further confirming the statistical significance of the finding.

The paired t-test results for individual LLM evaluation dimensions confirm the pattern observed in the effect sizes. Conciseness shows the most significant difference ($t=-3.58$, $p=0.006$), followed by LLM-Friendliness ($t=-3.42$, $p=0.007$), Completeness ($t=-3.08$, $p=0.014$), and Clarity ($t=-3.01$, $p=0.015$). Technical Accuracy shows the least significant difference ($t=-0.78$, $p=0.455$), which is not statistically significant at the 0.05 level. This pattern suggests that the LLM evaluator perceives the most substantial improvements in dimensions related to information presentation and formatting, while technical accuracy remains consistently high across both documentation types.

The largest effects are observed for Conciseness ($d=-1.347$) and LLM-Friendliness ($d=-1.371$), suggesting that machine-

optimized documentation is perceived as substantially more concise and better suited for LLM consumption. The Clarity ($d=-1.002$) and Completeness ($d=-0.964$) dimensions also show large effects, indicating that LLM evaluators perceive machine documentation as clearer and more comprehensive.

Technical Accuracy shows the smallest effect ($d=-0.245$), suggesting that both human and machine documentation maintain similar levels of technical correctness. This is expected, as both versions describe the same technical systems and APIs.

TABLE IV: Platform-by-Platform LRI Comparison

Platform	Human LRI	Machine LRI	Delta	Direction
Stripe	55.00	80.00	+25.00	Machine
FastAPI	87.00	100.00	+13.00	Machine
Hono	73.33	87.00	+13.67	Machine
Cursor	75.00	100.00	+25.00	Machine
Supabase	56.00	82.00	+26.00	Machine
GitHub	55.00	81.25	+26.25	Machine
Vercel	54.00	42.00	-12.00	Human
Cloudflare	63.00	80.00	+17.00	Machine
React	72.00	85.00	+13.00	Machine
LangChain	50.00	50.00	0.00	Tie

Delta = Machine LRI - Human LRI. Positive values indicate machine documentation preference.

Platform-level analysis reveals that 8 out of 10 platforms (80%) show higher LLM preference for machine-optimized documentation. Vercel is the notable exception, with human documentation scoring higher (54.00 vs. 42.00). This exception may be attributed to Vercel’s machine documentation being particularly link-heavy with minimal prose content, reducing its perceived completeness and clarity. LangChain shows no difference (50.00 vs. 50.00), suggesting that both versions are equally optimized for LLM consumption.

FastAPI and Cursor achieve perfect LRI scores (100.00) for machine documentation, indicating optimal LLM-friendliness according to the evaluator. This may reflect the maturity of their llm.txt implementations and the comprehensiveness of their machine-optimized content.

The GitHub platform shows a substantial improvement from human (55.00) to machine (81.25) documentation, with a delta of +26.25 points. This is particularly notable given that GitHub’s documentation is extremely extensive and covers a vast array of features. The machine-optimized version successfully condenses this information into a format that the LLM evaluator finds substantially more useful. Similarly, Stripe improves from 55.00 to 80.00 (+25.00), and Supabase from 56.00 to 82.00 (+26.00), indicating that payment and backend-as-a-service platforms benefit significantly from machine-optimized documentation.

Cloudflare’s improvement from 63.00 to 80.00 (+17.00) and React’s from 72.00 to 85.00 (+13.00) suggest that infrastructure and frontend frameworks also benefit from machine optimization, though the effect sizes are smaller than for developer platforms. This may reflect that these platforms have more technically sophisticated audiences, and their human documentation is already relatively well-optimized for technical readers.

The Hono platform shows an improvement from 73.33 to 87.00 (+13.67), indicating that even relatively newer frameworks can benefit from machine-optimized documentation. The fact that Hono’s human documentation already scores relatively high (73.33) suggests that the framework’s documentation team has invested in quality, but the machine-

optimized version still provides additional value for LLM consumption.

The LangChain platform presents a unique case where both human and machine documentation achieve identical LRI scores of 50.00. This parity suggests that LangChain’s human documentation is already well-optimized for LLM consumption, possibly due to its AI-native audience and the documentation team’s awareness of LLM consumption patterns. The equal scores also indicate that the machine-optimized version does not provide additional value beyond the human-readable version, raising questions about the necessity of maintaining separate machine-optimized documentation when human documentation already achieves high LLM-friendliness. This case suggests that the readability-tokenization paradox may be less pronounced for platforms that naturally serve AI-literate audiences, and that the decision to implement llm.txt should be guided by the existing quality of human documentation rather than applied uniformly across all platforms.

D. Correlation Analysis

The Pearson correlation analysis between traditional readability metrics and LLM evaluation scores reveals no significant correlations ($p > 0.05$) across all tested pairs. The highest correlation is observed between Flesch Reading Ease and LLM-Friendliness ($r=0.357$, $p=0.312$), but this does not reach statistical significance.

This lack of correlation provides critical evidence that traditional readability metrics and LLM evaluation scores measure fundamentally different constructs. While Flesch Reading Ease and Flesch-Kincaid Grade Level were designed to predict human reading comprehension based on sentence complexity and syllable density, LLM evaluation scores capture semantic quality, information density, and machine-specific optimization factors that traditional metrics cannot assess.

E. The Readability-Tokenization Paradox

The findings provide empirical evidence for what is termed the “readability-tokenization paradox”: machine-optimized documentation that reduces token count and improves LLM performance simultaneously decreases traditional human readability metrics. This paradox is demonstrated through the inverse relationship between Flesch Reading Ease and LLM Readability Index:

- Human documentation: Higher FRE ($M=33.39$), Lower LRI ($M=64.03$)
- Machine documentation: Lower FRE ($M=-58.18$), Higher LRI ($M=78.72$)

This paradox has significant implications for documentation strategy. Technical writers and documentation platform developers must balance the competing demands of human readability and machine optimization. The current trend toward llm.txt adoption suggests that the industry prioritizes machine optimization, potentially at the expense of human readability. However, this prioritization may not be optimal if the ultimate goal is to serve both human developers and AI assistants.

The paradox also raises questions about the evolving nature of technical communication. As LLMs increasingly mediate human access to technical information, the distinction between

“human-readable” and “machine-readable” documentation becomes increasingly blurred. When a developer asks an AI assistant about a library’s API, the assistant consumes the machine-optimized documentation but presents it in a human-friendly format. In this scenario, the LLM acts as a translator between machine-optimized source material and human-readable output, potentially rendering traditional human-readable documentation less critical for the end user.

F. Implications and Interpretation

The results suggest that the documentation landscape is undergoing a fundamental shift from human-centric to machine-optimized design. While traditional readability metrics remain valuable for assessing human comprehension, they are insufficient for evaluating documentation designed for dual human-machine consumption.

The large effect sizes for LLM evaluation dimensions (Cohen’s $d > -0.96$) indicate that machine-optimized documentation is not merely marginally better but substantially superior from an LLM perspective. This has practical implications for documentation teams: investing in llm.txt optimization can significantly improve LLM performance and user experience when documentation is consumed through AI assistants. The effect sizes suggest that the improvement is practically significant, not just statistically significant, meaning that real-world users will notice meaningful differences in LLM response quality.

The Vercel exception provides a particularly instructive case. Vercel’s human documentation scored higher (LRI=54.00) than its machine documentation (LRI=42.00), suggesting that the machine-optimized version was not effectively implemented for that platform. This exception highlights that the readability-tokenization paradox is not universal; it depends on implementation quality and content characteristics. Platforms that invest in high-quality machine-optimized documentation reap the benefits of improved LLM performance, while those with poorly implemented machine documentation may actually degrade both human and machine readability.

The outliers in React and LangChain (FRE=-430.09 and -234.73 respectively) require further interpretation. These extreme negative values suggest that the machine-optimized documentation for these platforms contains extremely dense, technical content with minimal explanatory prose. React’s documentation, for instance, is heavily code-centric, with extensive API reference material that naturally scores poorly on traditional readability metrics. However, the LLM evaluation scores for these platforms (React: LRI=85.00, LangChain: LRI=50.00) suggest that LLMs can still process this dense content effectively, though LangChain’s machine documentation did not achieve the high LLM scores that other platforms did.

The lack of correlation between traditional metrics and LLM scores (all $p > 0.05$) provides the most theoretically significant finding. This lack of correlation suggests that the two measurement systems are orthogonal: traditional metrics capture surface-level linguistic complexity (sentence length, syllable count), while LLM scores capture semantic and pragmatic quality (clarity, completeness, conciseness, LLM-friendliness). This orthogonality has important methodological

implications: researchers studying documentation quality cannot rely solely on traditional metrics and must incorporate machine-centric evaluation methodologies.

The strongest correlation observed, though still not statistically significant, is between Token-to-Word Ratio and LLM-Friendliness ($r=0.564$, $p=0.090$). This near-significant correlation suggests that token efficiency, a purely computational metric, may have some relationship with LLM-perceived quality. However, the correlation is driven by a few platforms with extreme token ratios (React: 5.57, LangChain: 5.43), and when these outliers are removed, the correlation drops to $r=0.312$, $p=0.412$. This sensitivity to outliers indicates that token efficiency alone is not a reliable predictor of LLM-perceived quality, and that other factors (such as content structure, clarity, and completeness) play more important roles.

The correlation between Flesch Reading Ease and LLM-Friendliness ($r=0.357$, $p=0.312$) shows a weak positive trend, suggesting that documentation that is easier for humans to read may also be somewhat more friendly to LLMs. However, this relationship is not strong enough to be statistically significant, and the direction of the correlation is opposite to what the readability-tokenization paradox would predict. This paradoxical finding may be explained by the fact that both human-readable and machine-optimized documentation can achieve high LLM-Friendliness scores if they are well-structured and clear, regardless of their traditional readability scores.

The correlation between token-to-word ratio and LLM scores ($r=0.564$, $p=0.090$) approaches statistical significance, suggesting that token efficiency may be one bridge between traditional metrics and LLM performance. While not conclusive at the 0.05 threshold, this trend indicates that documentation that uses fewer tokens per word (i.e., more common, less fragmented vocabulary) may be perceived more favorably by LLMs. This finding aligns with research on tokenization efficiency and suggests that vocabulary choice, not just sentence structure, influences machine readability.

G. Practical Implications for Documentation Teams

The findings provide actionable guidance for technical documentation teams. First, teams should invest in creating high-quality llm.txt files alongside their human-readable documentation. The significant LLM evaluation improvements (mean LRI difference of +14.69 points) suggest that this investment yields measurable improvements in LLM performance. Second, teams should not abandon human-readable documentation entirely, as the Vercel exception demonstrates that poorly implemented machine documentation can degrade both human and machine readability.

Third, the lack of correlation between traditional metrics and LLM scores suggests that documentation teams should use both human and machine evaluation metrics to assess documentation quality. Relying solely on Flesch Reading Ease or Grade Level may miss critical aspects of machine readability, while relying solely on LLM evaluation may miss aspects of human comprehension. A dual-metric approach provides the most comprehensive assessment.

Fourth, the extreme variance in traditional metrics for machine documentation (SD=155.28 for FRE) suggests that machine-optimized documentation is not uniform across

platforms. Teams should aim for consistent quality in their machine-optimized documentation, avoiding the extreme density that produces extremely negative readability scores. The ideal machine-optimized documentation balances conciseness with sufficient explanatory context to maintain readability for both humans and machines.

H. Theoretical Implications for Technolinguistics

The readability-tokenization paradox identified in this study contributes to the emerging field of technolinguistics by demonstrating that the relationship between human and machine language processing is not simply one of alignment or opposition. The paradox suggests that optimization for one type of processor (human or machine) can degrade performance for the other, but the degradation is not symmetric. Machine-optimized documentation degrades traditional human readability metrics significantly, but human-readable documentation does not necessarily degrade LLM performance (as evidenced by the Vercel exception, where human documentation scored higher on LLM evaluation).

This asymmetry suggests that the current generation of LLMs is more adaptable to diverse text formats than human readers are. LLMs can process dense, technical, poorly structured text and still extract meaningful information, while humans require clear structure, appropriate vocabulary, and sufficient context. This adaptability has implications for the future of technical communication: as LLMs improve, the pressure to optimize documentation for human readability may decrease, potentially leading to a bifurcation where documentation exists in two parallel forms (human-optimized and machine-optimized) rather than a single dual-purpose format.

The study also introduces the LLM Readability Index (LRI) as a standardized metric for machine readability assessment. While preliminary, this metric provides a framework for future research to compare documentation quality across platforms and evaluate the effectiveness of machine-optimized documentation standards. The LRI normalizes the 1-5 Likert scale to a 0-100 range, enabling direct comparison with traditional metrics and facilitating cross-platform analysis.

V. CONCLUSION

This study provides empirical evidence for the readability-tokenization paradox in technical documentation. Through analysis of ten platforms using both traditional readability metrics and a novel LLM-as-a-Judge methodology, it is demonstrated that machine-optimized documentation (llm.txt) significantly improves LLM-perceived quality while generally decreasing traditional human readability metrics.

The key findings are:

- 1) **RQ1:** Machine-optimized documentation exhibits significantly different traditional readability metrics, with Flesch Reading Ease showing mixed results due to outlier sensitivity, but Flesch-Kincaid Grade Level consistently indicating higher reading difficulty (+10.2 grade levels).
- 2) **RQ2:** LLM-as-a-Judge scores significantly favor machine-optimized documentation ($p=0.005$), with large effect sizes across all evaluation dimensions (Cohen's d ranging from -0.964 to -1.371).

- 3) **RQ3:** Traditional readability metrics are insufficient for evaluating LLM-optimized documentation, as evidenced by the lack of significant correlation between traditional metrics and LLM scores (all $p > 0.05$).

These findings contribute to the emerging field of technolinguistics by quantifying the tension between human and machine optimization in documentation design. The LLM Readability Index (LRI) introduced in this study provides a standardized metric for future research comparing human and machine documentation quality.

The study's methodological contribution is equally significant. The automated documentation auditing tool developed for this study provides a reproducible framework for large-scale documentation analysis. The tool's modular architecture, comprehensive test suite, and caching mechanisms make it suitable for longitudinal studies tracking documentation quality over time. The integration of multiple evaluation methodologies (traditional metrics, LLM evaluation, statistical analysis) within a single pipeline demonstrates the feasibility of comprehensive documentation assessment at scale.

From a practical standpoint, the findings suggest that documentation teams should adopt a dual-format strategy: maintain human-readable HTML documentation for direct developer consumption while simultaneously investing in high-quality llm.txt files for LLM-mediated consumption. The significant improvement in LLM evaluation scores (mean LRI increase of +14.69 points) demonstrates that this investment yields measurable returns in terms of AI assistant performance. The Vercel exception, however, cautions that poorly implemented machine documentation can actually degrade both human and machine readability, emphasizing the importance of quality in machine-optimized documentation.

The theoretical implications of the readability-tokenization paradox extend beyond documentation to other forms of technical communication. As LLMs become increasingly central to information retrieval and processing, the tension between human and machine optimization will likely manifest in other domains: academic papers, legal documents, medical records, and scientific reports. The methodology developed in this study provides a template for investigating these tensions in other contexts, contributing to the broader field of human-machine communication.

A. Limitations

This study has several limitations that should be acknowledged. The small sample size ($n=10$ platforms) limits its generalizability; larger-scale studies are needed to confirm these findings. The English-only corpus restricts applicability to non-English documentation. The single LLM evaluator (nvidia/nemotron-nano-12b-v2-v1) may introduce model-specific bias; multi-model consensus evaluation would strengthen validity, as demonstrated by Chen et al. [23] in scientific programming contexts. Finally, the purposive sampling strategy limits representativeness across all technical documentation genres.

B. Future Work

Future research should expand the corpus to 50+ platforms, include non-English documentation, and employ multi-model LLM evaluation for consensus scoring. Comparative analysis

with Markdown-formatted documentation and user studies examining actual human reading performance would provide additional validation. The development of hybrid metrics that simultaneously capture human and machine readability remains an important research direction.

Specifically, the following research directions are recommended: (1) Cross-platform validation: Replicate this study on a larger corpus of 50+ platforms to confirm the generalizability of the readability-tokenization paradox. (2) Multi-model evaluation: Employ 3-5 different LLM evaluators (e.g., GPT-4, Claude, Llama, Gemini) to establish consensus scores and assess model-specific biases. (3) Non-English documentation: Extend the methodology to Chinese, Japanese, and European languages to assess cross-linguistic applicability. (4) Longitudinal analysis: Track documentation quality over time as platforms evolve their llm.txt implementations. (5) User studies: Conduct controlled experiments with human developers to measure actual comprehension and task completion times using human vs. machine-optimized documentation. (6) Hybrid metrics: Develop composite metrics that combine traditional readability scores with LLM evaluation scores to create a unified documentation quality index. (7) Economic analysis: Quantify the cost-benefit tradeoffs of maintaining dual documentation formats, considering API costs, developer time, and user satisfaction. (8) Integration studies: Investigate how llm.txt documentation affects the performance of downstream AI applications, such as code generation and technical support chatbots.

ACKNOWLEDGMENT

The author acknowledges the use of AI tools for statistical analysis, visualization, and literature review synthesis. The LLM-as-a-Judge evaluation was conducted using the nvidia/nemotron-nano-12b-v2-v1 model via the OpenRouter API. This research was submitted to CODHES 2026 [?]. The author maintains full responsibility for research design, data interpretation, and conclusion formulation.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are openly available in the GitHub repository at <https://github.com/yehezkielgunawan/codhes-2026>, including raw scraped texts, statistical analysis scripts, and generated charts.

APPENDIX: REPRESENTATIVE DOCUMENTATION SAMPLES

To illustrate the structural and stylistic differences between the two documentation corpora, representative excerpts from two platforms are provided below. The human-readable documentation samples were extracted after automated cleaning. The machine-optimized documentation samples are the raw llm.txt or llms.txt files as retrieved from each platform.

A.1 FastAPI

Human-readable documentation excerpt:

```
### GET STARTED
* Quick Start
  * Tutorial: Tic-Tac-Toe
  * Thinking in React
* Installation
  * Creating a React App
  * Build a React App from Scratch
  * Add React to an Existing Project
* Setup
  * Editor Setup
```

```
* Using TypeScript
* React Developer Tools
* React Compiler
* Introduction
* Installation
* Incremental Adoption
* Debugging and Troubleshooting
### LEARN REACT
* Describing the UI
* Your First Component
* Importing and Exporting Components
* Writing Markup with JSX
* JavaScript in JSX with Curly Braces
* Passing Props to a Component
* Conditional Rendering
* Rendering Lists
* Keeping Components Pure
* Your UI as a Tree
```

Machine-optimized documentation excerpt:

```
### Add Summary and Description to FastAPI
### Path Operations

Source: github.com/fastapi/fastapi/...

Provide a concise summary and a more detailed
description for a path operation, improving its
clarity in the OpenAPI documentation.

```python
from fastapi import FastAPI

app = FastAPI()

@app.post("/items/",
 summary="Create an item",
 description="Create an item with all
 the information...",
)
async def create_item(name: str):
 return {"name": name}
```

### Define FastAPI Path Operation Description
### from Docstring

Source: github.com/fastapi/fastapi/...

Use a function's docstring to provide a multi-line
description for a path operation. Markdown in
docstrings is supported.

```python
from fastapi import FastAPI

app = FastAPI()

@app.post("/items/")
async def create_item(name: str):
 """
 Create an item with all the information:
 - **name**: Name of the item
 - **description**: A long description
 - **price**: Price of the item
 """
 return {"name": name}
```
```

A.2 React

Human-readable documentation excerpt:

```
Copyright (c) Meta Platforms, Inc
no uwu plz
Logo by @sawaratsukil004

Search
The library for web and native user interfaces
## Create user interfaces from components
React lets you build user interfaces out of
individual pieces called components. Create your
own React components like , , and . Then combine
them into entire screens, pages, and apps.

### Video.js
My video Video description
Whether you work on your own or with thousands
of other developers, using React feels the same.
It is designed to let you seamlessly combine
components written by independent people, teams,
and organizations.

## Write components with code and markup
React components are JavaScript functions. Want
to show some content conditionally? Use an
statement. Displaying a list? Try array .
Learning React is learning programming.

### VideoList.js
## 3 Videos
First video Video description
Second video Video description
Third video Video description
```

Machine-optimized documentation excerpt:

```
# React Documentation

> The library for web and native user interfaces.

## Learn React
```

```

### GET STARTED

#### Quick Start
- [Quick Start](...)
- [Tutorial: Tic-Tac-Toe](...)
- [Thinking in React](...)

#### Installation
- [Installation](...)
- [Creating a React App](...)
- [Build a React App from Scratch](...)
- [Add React to an Existing Project](...)

#### Setup
- [Setup](...)
- [Editor Setup](...)
- [Using TypeScript](...)
- [React Developer Tools](...)

```

REFERENCES

- [1] J. Howard. (2024, September 3) The //lms.txt file: A proposal to standardise on using an //lms.txt file to provide information to help LLMs use a website at inference time. [Online]. Available: <https://llmstxt.org/>
- [2] AnswerDotAI. (2024) llms-txt. GitHub. [Online]. Available: <https://github.com/AnswerDotAI/llms-txt>
- [3] (2024) llmstxt.site. [Online]. Available: <https://llmstxt.site/>
- [4] (2024) directory.llmstxt.cloud. [Online]. Available: <https://directory.llmstxt.cloud/>
- [5] R. Flesch, "A new readability yardstick," *Journal of Applied Psychology*, vol. 32, no. 3, pp. 221–233, 1948.
- [6] J. P. Kincaid, R. P. Fishburne, R. L. Rogers, and B. S. Chissom, "Derivation of new readability formulas (automated readability index, fog count and flesch reading ease formula) for navy enlisted personnel," Naval Air Station Memphis, Research Branch Report 8-75, 1975. [Online]. Available: <https://apps.dtic.mil/sti/pdfs/ADA006655.pdf>
- [7] J. C. Redish, "Readability formulas have even more limitations than Klare discusses," *ACM Journal of Computer Documentation*, vol. 24, no. 3, pp. 132–137, 2000.
- [8] G. McClure, "Readability formulas: Useful or useless? (an interview with J. Peter Kincaid)," *IEEE Transactions on Professional Communication*, vol. 30, no. 1, pp. 12–15, 1987.
- [9] J. P. Kincaid, R. Braby, and J. Mears, "Electronic authoring and delivery of technical information," *Journal of Instructional Development*, vol. 11, no. 2, pp. 8–13, 1988.
- [10] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL 2016)*, Berlin, Germany, 2016, pp. 1715–1725.
- [11] OpenAI. (2023) tiktoken: A fast BPE tokeniser for use with OpenAI's models. GitHub. [Online]. Available: <https://github.com/openai/tiktoken>
- [12] Z. Ankner, A. Bertsch, G. Neubig, and M. R. Gormley, "The "·" tokenization," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
- [13] S. Ding, A. Renduchintala, and K. Duh, "A call for prudent choice of subword merge operations in neural machine translation," in *Proceedings of MT Summit 2019*, 2019. [Online]. Available: <https://aclanthology.org/W19-6608/>
- [14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 11, 2023.
- [15] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*, 2023, pp. 18 020–18 036.
- [16] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Proceedings of the Neural Information Processing Systems (NeurIPS) 2023 Datasets and Benchmarks Track*, 2023.
- [17] K. Horikawa, K. Horikawa, Y. Kashiwa, H. Uwano, and H. Iida, "Do AI agents really improve code readability?" in *Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26)*, 2026. [Online]. Available: <https://arxiv.org/abs/2603.13723>
- [18] I. R. d. S. Simoes and E. Venson, "Measuring how changes in code readability attributes affect code quality evaluation by large language models," in *Proceedings of the XXXIX Brazilian Symposium on Software Engineering*, 2025.
- [19] C. Diggs, M. Doyle, A. Madan, S. Scott, E. Escamilla, J. Zimmer, N. Nekoo, P. Ursino, M. Bartholf, Z. Robin, A. Patel, C. Glasz, W. Macke, P. Kirk, J. Phillips, A. Sridharan, D. Wendt, S. Rosen, N. Naik, J. F. Brunelle, and S. Thaker, "Leveraging LLMs for legacy code modernization: Challenges and opportunities for LLM-Generated documentation," in *Proceedings of the LLM4Code Workshop at ICSE 2025*, 2024. [Online]. Available: <https://arxiv.org/abs/2411.14971>
- [20] Textstat. (2023) textstat: Calculate statistical features from text. Python package. [Online]. Available: <https://github.com/shivam5992/textstat>
- [21] Mozilla. (2024) Readability: A standalone version of the readability library used for Firefox reader view. Mozilla. [Online]. Available: <https://github.com/mozilla/readability>
- [22] (2024) Openrouter: A unified interface for LLMs. [Online]. Available: <https://openrouter.ai/>
- [23] A. Chen, C. Wong, B. Sharif, and A. Peruma, "Exploring code comprehension in scientific programming: Preliminary insights from research scientists," in *Proceedings of the 33rd IEEE/ACM International Conference on Program Comprehension (ICPC 2025)*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.10037>